Bootloader, Firmware ve Gömülü Yazılım Güncelleme Mimarisi: Teknik Eğitim Raporu

1. Giriş ve Temel Mimariler (Firmware vs. Bootloader)

1.1. Firmware (Gömülü Cihaz Yazılımı)

Firmware (Gömülü Cihaz Yazılımı), bir elektronik cihazın donanım bileşenlerini doğrudan denetleyen, alt seviye mantıksal kararları alan ve fiziksel işlevleri icra eden özelleşmiş yazılım katmanıdır. Masaüstü veya sunucu ortamlarında çalışan genel amaçlı uygulama yazılımlarından ve işletim sistemlerinden temel farkı; soyutlanmış üst seviye bir grafik kullanıcı arayüzüne ihtiyaç duymaksızın doğrudan mikromimari seviyesindeki donanım saklayıcıları (registers) ve bellek alanları ile etkileşime girmesidir.

Gömülü sistemler mimarisinde iki temel işlem birimi öne çıkar:

- **CPU (Central Processing Unit / Merkezi İşlem Birimi):** Sistemdeki komutları sırayla işleyen, aritmetiksel ve mantıksal operasyonları yürüten ana işlem çekirdeğidir.
- **MCU (Microcontroller Unit / Mikrodenetleyici):** CPU çekirdeğini; kalıcı/geçici bellek birimleri (Flash, RAM) ve çevre birimleri (kesme denetleyicileri, zamanlayıcılar, haberleşme portları) ile tek bir silikon entegre devre (çip) üzerinde birleştiren sistemdir.

Firmware, bir MCU üzerinde yürütülürken çevre birimlerinin (peripherals) sürülmesi, sensör verilerinin analog-dijital dönüştürücüler aracılığıyla okunması, motor sürücü sinyallerinin üretilmesi, seri haberleşme protokollerinin yönetilmesi ve donanımsal hata durumlarının işlenmesi gibi doğrudan fiziksel dünyaya temas eden görevleri üstlenir. Dolayısıyla firmware, sadece derlenmiş bir ikili dosya değil, donanıma hayat veren mimari bir bileşendir.

1.2. Bootloader (Önyükleyici)

Bootloader (Önyükleyici), bir sistem açıldığında veya sıfırlandığında (reset) ilk olarak çalışan; donanım ortamını hazırlayan, bir sonraki yazılım imgesinin bütünlüğünü ve yetkisini doğrulayan, ana uygulamanın bellek konumunu belirleyerek çalışmayı o noktaya devreden özel nitelikli bir yazılımdır.

> **Kavramsal İlke:** Bootloader bağımsız bir yazılım kategorisi değil, işlevsel olarak önyükleme amacıyla özelleştirilmiş bir firmware türüdür (**önyükleyici firmware**). Sistem mimarisinde "önyükleyici firmware" ile cihazın nihai saha görevini üstlenen "uygulama firmware'i" (Application Firmware) net sınırlarla ayrılmıştır.

Tüm mikrodenetleyici tasarımlarında geliştirici tarafından yazılmış özel bir kullanıcı önyükleyicisi bulunması zorunlu değildir. Bir MCU, sıfırlama sonrasında doğrudan dahili Flash bellekteki ana uygulama yazılımını da başlatabilir. Donanım seviyesinde iki temel önyükleyici yapısı bulunmaktadır:

1. **Fabrika ROM Yükleyicisi (Factory ROM Bootloader):** Yonga üreticisi tarafından mikrodenetleyicinin üretimi esnasında dökümhanede salt okunur bellek alanına (ROM)

kazınan, silinemeyen ve değiştirilemeyen koddur.

2. **Flash Tabanlı Özel Önyükleyici (Flash-Based Custom Bootloader):** Geliştirici mühendisler tarafından tasarlanan, cihazın kalıcı Flash belleğine yazılan, saha şartlarına göre esnek güncelleme ve güvenlik mantıkları içeren yazılımdır.

Kavram	Temel Görev ve Özellikler
Uygulama Firmware'i (Application Firmware)	Cihazın nihai işlevselliğini (sensör verisi işleme, kablosuz haberleşme, motor kontrolü, kullanıcı arayüzü vb.) yürüten ana yazılımdır.
Önyükleyici Firmware (Bootloader Firmware)	Reset sonrasında çalışan, donanım doğrulaması yapan, bellek sektörlerini yöneten ve ana uygulamayı güvenle başlatan altyapı yazılımıdır.

1.3. Bellek Teknolojileri ve Donanım Katmanları

Gömülü sistem mimarilerinde firmware kodlarının ve verilerin konumlandırıldığı üç ana bellek teknolojisi kullanılır:

- **ROM (Read-Only Memory / Salt Okunur Bellek):** Üretim aşamasında içeriği sabitlenen, güç kesilse de veriyi koruyan ve elektriksel olarak normal şartlarda değiştirilemeyen kalıcı bellek birimidir.
- **Flash (Elektriksel Silinebilir Kalıcı Bellek):** Bloklar, sektörler (sector) veya sayfalar (page) halinde elektriksel olarak silinip yeniden programlanabilen, güç kesildiğinde içeriğini kaybetmeyen (non-volatile) kalıcı bellektir.
- **RAM (Random Access Memory / Rastgele Erişimli Çalışma Belleği):** İşlemcinin okuma ve yazma işlemlerini yüksek hızda gerçekleştirdiği uçucu (volatile) çalışma belleğidir. Güç kesildiğinde içerik kaybolur; değişkenler, yığın (stack) ve dinamik bellek (heap) RAM üzerinde tutulur.

Firmware kodları fiziksel olarak Flash veya ROM içerisinde saklanır. İşlemci mimarisine bağlı olarak kodlar doğrudan Flash üzerinden yürütülebilir (XIP - Execute-in-Place / Yerinde Yürütme - Kodu Flash'tan kopyalamadan doğrudan yürütme) ya da açılışta RAM'e aktarılarak RAM üzerinden yüksek hızda çalıştırılabilir.

> **Mühendislik Yanılgısı:** Bir firmware ikili dosyasının (binary file) diske kaydedilen boyutu, cihazın ihtiyaç duyduğu RAM miktarını göstermez. Dosya boyutu, Flash bellekte kaplanan sabit kod ve sabit veri alanını temsil eder; RAM gereksinimi ise programın çalışma zamanında ihtiyaç duyduğu **.data** (başlangıç değeri atanmış değişkenler), **.bss** (başlangıç değeri atanmamış değişkenler) bölümleri ile dinamik yığın/bellek ayırmalarıyla belirlenir.

---

2. Derleme Zinciri, Dosya Biçimleri ve Bellek Haritası

2.1. Kaynak Koddan İkili Görüntüye (Source Code to Firmware Image)

İnsan tarafından okunabilir C/C++ kaynak kodlarının donanım üzerinde icra edilebilir bir firmware imgesine dönüşmesi, adımlı bir derleme hattı (compilation pipeline) üzerinden gerçekleşir:

Kaynak Kod (Source Code) → [Derleyici (Compiler)] Amaç Kod (Object Code) → [Bağlayıcı (Linker)] Yazılım Görüntüsü (Firmware Image)

1. **Kaynak Kod (Source Code):** Geliştirici tarafından yazılan, yüksek seviyeli programlama talimatları içeren metin dosyalarıdır.
2. **Derleyici (Compiler):** Kaynak kod dosyalarını işleyerek hedef işlemci mimarisinin (örneğin Arm Cortex-M veya RISC-V) makine komutlarına dönüştürür ve amaç dosyalarını (**.o** / **.obj**) üretir.
3. **Bağlayıcı (Linker):** Ayrık amaç dosyalarını ve harici kütüphaneleri birleştirir, sembol adreslerini çözer ve kod bloklarını mikrodenetleyicinin fiziksel bellek haritasına bağlayıcı metni (linker script) kurallarına göre yerleştirir.
4. **Yazılım Görüntüsü (Firmware Image):** Kalıcı belleğe yazılacak olan; makine kodlarını, sabit verileri, başlık (header) metadatalarını ve doğrulama anahtarlarını barındıran ikili yapıdır.

2.2. Dosya Biçimlerinin Karşılaştırılması: ELF, BIN ve Intel HEX

Derleme zinciri çıktısı olarak farklı mühendislik ihtiyaçlarına yanıt veren üç ana dosya biçimi kullanılır:

- **ELF (Executable and Linkable Format / Yürütülebilir ve Bağlanabilir Dosya Biçimi):** İçerisinde bellek kesimlerini, sembol tablolarını, hata ayıklama (debug) sembollerini barındıran kapsamlı bir yapıdır.
- **BIN (Binary / Ham İkili Dosya):** Herhangi bir adres bilgisi veya başlık içermeyen, kalıcı belleğe birebir kopyalanacak olan ham makine kodu dizisidir.
- **Intel HEX (Hexadecimal Text / Onaltılık Metin Biçimi):** Satır bazlı ASCII metin yapısında olan, veri baytlarının yanı sıra hedef bellek adreslerini ve kontrol toplamlarını (checksum) barındıran formattır.

Dosya Biçimi	İçerik Mimarisi	Adres Bilgisi İçerir mi?	Kullanım Alanı
ELF	Başlıklar, kod kesimleri, sembol tabloları, debug verileri.	Evet (Bölüm bazlı ayrıntılı adresler)	Hata ayıklama (debugging) ve simülasyon ortamları.
BIN	Sadece ham makine kodları ve veri baytları.	**Hayır**	Bootloader ile OTA/IAP güncellemeleri, doğrudan Flash yazma.
Intel HEX	ASCII tabanlı adres, veri ve kontrol satırları.	Evet (Satır bazlı adres kayıtları)	Harici programlayıcılar ve fabrika ROM yükleyicileri.

> **Kritik Detay:** Ham BIN dosyaları bünyesinde adres verisi taşımadığı için, yükleyici yazılıma veya programlayıcı araca verinin Flash belleğin hangi başlangıç adresinden itibaren yazılacağının harici olarak bildirilmesi zorunludur.

2.3. Linker Script, Bellek Haritası (Memory Map) ve Vektör Tablosu

Yazılımın bellek üzerinde hatasız konumlandırılması ve çalışması şu mimari elemanlarla sağlanır:

- **Bellek Haritası (Memory Map):** İşlemcinin adresleyebildiği tüm adres uzayının (Flash, RAM, çevre birim saklayıcıları) haritasıdır.
- **Linker Script (Bağlayıcı Metni):** Bağlayıcıya, derlenmiş koda ait kesimlerin (**.text**, **.rodata**, **.data**, **.bss**) bellek üzerindeki hangi kesin adres aralıklarına yerleştirileceğini belirten konfigürasyon dosyasıdır.
- **Vector Table (Vektör Tablosu):** İşlemcinin sıfırlama, sistem hataları (faults) ve donanım kesilmeleri (interrupts) durumunda atlayacağı işleyici (handler) fonksiyonların adreslerini barındıran dizilimdir.
- **Reset Vector (Reset Vektörü):** Vektör tablosunun ikinci girdisinde yer alan ve işlemci resetlendiğinde yürütülecek ilk komutun adresini belirten göstergedir.
- **Stack (Yığın):** Fonksiyon çağrıları, yerel değişkenler ve kesme durumlarında yazmaç saklama işlemleri için kullanılan RAM alanıdır.
- **Startup Code (Başlangıç Kodu):** İşlemci resetlendiğinde **main()** fonksiyonundan önce çalışan; yığın göstergesini (Stack Pointer) yükleyen, **.data** kesimini Flash'tan RAM'e kopyalayan ve **.bss** kesimini sıfırlayan hazırlık programıdır.

Uygulama yazılımı Flash üzerinde varsayılan **0x00000000** adresi yerine bir bootloader offset'ine (örneğin **0x00010000**) yerleştirildiğinde, kesmelerin doğru işleyicilere yönlendirilebilmesi için **VTOR (Vector Table Offset Register / Vektör Tablosu Offset Saklayıcısı)** yönlendirmesinin bootloader tarafından yapılması şarttır.

---

3. Resetten Uygulamaya Açılış Zinciri (Boot Execution Chain)

3.1. Evrensel Açılış Adımları ve Çalışma Mantığı

Bir mikrodenetleyicinin ilk güç alması veya resetlenmesinden ana uygulamanın **main()** fonksiyonuna ulaşmasına kadar geçen kavramsal açılış zinciri şu adımlardan oluşur:

1. **Güç Uygulanması / Reset Tetiklenmesi (Power/Reset Trigger):** Besleme voltajının kararlı seviyeye ulaşması veya donanımsal reset hattının serbest bırakılması.
2. **Donanım Başlangıç Adresinin ve Reset Vektörünün Okunması:** CPU donanımsal olarak tanımlı ilk adresten Başlangıç Yığın Göstergesini ve Reset Vektörünü okur.
3. **İlk Kodun (Boot ROM / Bootloader) Çalıştırılması:** Reset vektörünün işaret ettiği adresteki ilk kod (Boot ROM veya dahili Flash'taki özel bootloader) yürütülmeye başlanır.
4. **Saat (Clock) ve Bellek (RAM/Flash) Hazırlıkları:** İşlemci saat frekansları, dâhili/harici bellek denetleyicileri kararlı duruma getirilir.
5. **Uygulama Doğrulama ve Vektör Tablosu Yönlendirme (VTOR):** Başlatılacak uygulamanın bütünlüğü ve imzası doğrulanır. Kesme vektör tablosu adresi uygulamanın offset adresine (VTOR) güncellenir.
6. **Startup Code Yürütülmesi ve Uygulama main() Fonksiyonuna Atlama:** Uygulamanın başlangıç kodu RAM değişkenlerini hazırlar ve kontrol ana uygulamanın

main() fonksiyonuna devredilir.

3.2. Donanıma Özel vs. Evrensel Açılış Kuralları

Açılış ilkesi evrensel olmakla birlikte, başlangıç kodunun seçim mekanizması üreticiye ve donanım mimarisine göre farklılık gösterir. Gömülü mimarilerde donanım açılış modu seçimi üç temel yöntemle gerçekleştirilir:

1. **Fiziksel Pin Seviyesi Örneklemesi (BOOT Pin Sampling):** Sıfırlama sinyalinin kalktığı anda (rising edge) işlemci, belirli fiziksel pinlerin (**BOOT0 / BOOT1**) voltaj seviyelerini kilitler (latch). Örneğin pin kombinasyonuna göre kodun Dahili Flash (**0x08000000**), Sistem Hafızası ROM (**0x1FFF0000**) veya Dahili RAM üzerinden başlatılacağı donanımsal olarak seçilir.
2. **eFuse (Elektronik Sigorta) ve Donanımsal Konfigürasyon Bitleri:** Yonga üzerindeki tek sefer programlanabilir (OTP) eFuse hücreleri, açılış varsayılanlarını donanımsal olarak sabitler. Güvenlik odaklı çiplerde Boot ROM, ilk olarak eFuse bitlerini okuyarak Secure Boot süreçlerini başlatır.
3. **Yazılımsal Seçenek Baytları (Option Bytes):** Flash belleğin özel ayrılmış konfigürasyon alanlarında tutulan option byte yapıları, pin durumu ne olursa olsun donanım önyükleme adresinin yazılımsal olarak sistem hafızasına veya ana Flash'a kilitlenmesini sağlar.

---

4. Yükleyici Sınıflandırması: ROM Loader, Uygulama Yükleyicisi ve U-Boot

4.1. Üretici Fabrika ROM Yükleyicisi (Factory ROM Bootloader)

Üretici tarafından yonganın silinemez sistem belleğine (System Memory) yerleştirilen koddur.

- **STMicroelectronics AN2606 (System Memory Boot Mode):** STM32 serisinde BOOT pinlerinin kombinasyonu ile devreye giren fabrikasyon önyükleme modudur.
- **STMicroelectronics AN3155 (USART Bootloader Protocol):** Seri hat üzerinden çipe bağlanarak dahili Flash belleği programlamayı sağlayan komut kümesi ve el sıkışma (handshake) protokolüdür.

Fabrika ROM yükleyicisi sayesinde çipte hiçbir kullanıcı yazılımı olmasa dahi, BOOT pinleri ayarlanarak UART veya USB üzerinden dahili Flash silinebilir ve sıfırdan programlanabilir.

4.2. Özel Uygulama Önyükleyicileri (Flash-Based Custom Application Bootloader)

Geliştiriciler tarafından yazılan ve dahili Flash belleğin ilk sektörlerine yerleştirilen önyükleyicilerdir. Fabrika ROM yükleyicilerinin sunduğu sabit protokollerin aksine; özel önyükleyiciler karmaşık şifreleme algoritmaları, özelleştirilmiş kablosuz ağ protokolleri, yedekli bellek alanları (slot yönetimi) ve gelişmiş geri alma (rollback) yetenekleri sunar.

4.3. MPU ve İşletim Sistemi Yükleyicileri: U-Boot Analizi

Gelişmiş mimarilerde işlemci ve bellek yapısı farklı bir boyuta taşınır:

- **MPU (Microprocessor Unit / Mikroişlemci Birimi):** Yüksek işlem gücüne sahip, genellikle harici bellek kullanan ve gelişmiş donanım birimlerine sahip işlemcilerdir.
- **OS (Operating System / İşletim Sistemi):** Donanım kaynaklarını yöneten ve süreçleri zamanlayan kompleks yazılım mimarisidir.
- **MMU (Memory Management Unit / Bellek Yönetim Birimi):** Sanal bellek adreslerini fiziksel bellek adreslerine dönüştüren donanım birimidir.
- **DRAM (Dynamic Random-Access Memory / Dinamik Rastgele Erişimli Bellek):** Yüksek kapasiteli harici geçici sistem belleğidir.
- **Linux Kernel:** İşletim sisteminin donanım soyutlama ve kaynak yönetimini üstlenen çekirdek katmanıdır.

MPU tabanlı sistemlerde (örneğin Microchip AT91 / SAMA serisi) bellek başlatma karmaşıklığı nedeniyle çok aşamalı önyükleme (multi-stage boot) mimarisi uygulanır [S07]:

1. Aşama: ROM Boot → 2. Aşama: AT91Bootstrap / SPL → 3. Aşama: U-Boot → Linux Kernel

1. **Birinci Aşama (ROM Boot):** İşlemci ROM'undaki kod çalışır, temel saatleri başlatır ve ikincil yükleyiciyi SRAM'e kopyalar.
2. **İkinci Aşama (AT91Bootstrap / SPL - Secondary Program Loader / İkincil Program Yükleyicisi):** Harici DRAM belleği başlatır ve üçüncü aşama yükleyiciyi (U-Boot) DRAM'e yükler.
3. **Üçüncü Aşama (U-Boot - Das U-Boot):** Donanım arabirimlerini hazırlar, gelişmiş komut satırını ve ağ servislerini çalıştırır, ardından Linux çekirdeğini başlatır.

U-Boot ortam değişkenleri ve komut yapısı [S07]:

- **setenv**: Değişken tanımlar veya günceller.
- **saveenv**: Değişkenleri kalıcı depolamaya (NAND/Flash/SD) kaydeder.
- **printenv**: Mevcut tüm ortam değişkenlerini listeler.
- **bootcmd**: Otomatik önyükleme geri sayımı dolduğunda yürütecek komut zincirini saklar.
- **bootargs**: Linux çekirdeğine aktarılacak komut satırı parametrelerini (konsol hızı, rootfs konumu vb.) tutar.
- **bootm** / **bootz**: Yapılandırılmış imge dosyalarından (uImage/zImage) işletim sistemini başlatır.

U-Boot, donanım mimarisini çekirdeğe aktarmak için **DTB (Device Tree Blob / Aygıt Ağacı İkili Bloğu)** veya tüm bileşenleri tek pakette birleştiren **FIT (Flattened Image Tree / Düzleştirilmiş Görüntü Ağacı)** formatını kullanır [S07].

4.4. Yükleyici Türleri Karşılaştırma Tablosu

Özellik / Kriter	Fabrika ROM Yükleyicisi	Özel Uygulama Bootloader'ı	U-Boot (MPU / Linux)
Konum	Üretici ROM (System Memory)	Dahili Flash Bellek	Harici Flash / NAND / SD

Özellik / Kriter	Fabrika ROM Yükleyicisi	Özel Uygulama Bootloader'ı	U-Boot (MPU / Linux)
Değiştirilebilirlik	Değiştirilemez (Salt Okunur)	Geliştirici tarafından güncellenebilir	Derlenebilir ve özelleştirilebilir
Ana Hedef	İlk fabrikasyon yükleme / Kurtarma	Uygulama güncelleme (OTA/IAP)	İşletim sistemi (Linux) başlatma
Karmaşıklık	Çok Düşük	Orta - Yüksek	Çok Yüksek

---

5. Programlama, Haberleşme Arayüzleri ve Güncelleme Yöntemleri

5.1. Donanım Arayüzleri: SWD, JTAG, UART, USB, SPI

Yazılımın cihaza aktarılmasında iki temel donanım arayüzü sınıfı ayırt edilmelidir:

1. **Donanımsal Hata Ayıklama ve Doğrudan Çip Programlama Arayüzleri:**
   - **SWD (Serial Wire Debug / Seri Tel Hata Ayıklama):** Arm mimarilerinde kullanılan, iki hat (SWCLK, SWDIO) üzerinden donanımsal hata ayıklama ve kod yükleme sağlayan protokoldür.
   - **JTAG (Joint Test Action Group / Ortak Test Eylem Grubu):** Çip seviyesinde test ve programlama sağlayan, standartlaştırılmış çok hatlı (TDI, TDO, TCK, TMS) arayüz mimarisidir.
2. **Seri Haberleşme Arayüzleri:**
   - **UART (Universal Asynchronous Receiver/Transmitter / Asenkron Seri Alıcı/Verici):** Asenkron seri veri aktarımı yapan donanım birimidir.
   - **USART (Universal Synchronous/Asynchronous Receiver/Transmitter / Senkron-Asenkron Seri Alıcı/Verici):** Senkron veya asenkron çalışabilen gelişmiş seri birimdir.
   - **USB (Universal Serial Bus / Evrensel Seri Veri Yolu):** Yüksek hızlı evrensel seri bağlantı standardıdır.
   - **SPI (Serial Peripheral Interface / Seri Çevre Birimi Arabirimi):** Tam çift yönlü (full-duplex) senkron seri haberleşme protokolüdür.

> **Mimarisi Ayrımı:** SWD ve JTAG, işlemci çekirdeğine ve bellek veri yoluna doğrudan fiziksel erişim sağlayan hata ayıklama arabirimleridir; harici bir programlayıcı donanım gerektirir. UART, USB ve SPI ise bootloader veya uygulama seviyesinde veri paketlerini taşımak için kullanılan haberleşme hatlarıdır.

5.2. Yazılımsal Güncelleme Mimarileri: IAP ve OTA

- **IAP (In-Application Programming / Uygulama İçinden Programlama):** Çalışmakta olan ana uygulamanın, gelen yeni yazılım paketlerini kabul ederek kalıcı belleğin belirlenen alanlarına yazması işlemidir [S03].

- **OTA (Over-The-Air / Kablosuz-Uzaktan Güncelleme):** Yazılım güncelleme paketlerinin Wi-Fi, Bluetooth veya hücresel ağlar gibi kablosuz ortamlar üzerinden cihaza ulaştırılması mimarisidir [S06].

IAP ve OTA süreçleri yalnızca bir dosyanın indirilmesinden ibaret değildir. Başarılı bir güncelleme mimarisi; indirme (download), staging (geçici depolama), doğrulama (verification), Flash silme/yazma (flash writing), boot durum kaydı güncellemesi ve uygulamaya geçiş (app handover) aşamalarının tamamını kapsar.

---

6. Güvenlik, Bütünlük, Doğrulama ve Kriptografik Ayrımlar

6.1. Bütünlük ve Kimlik Doğrulama Kavramları

Bir güncelleme mimarisinde güvenlik üç temel sütun üzerine kuruludur:

1. **Veri Bütünlüğü (Data Integrity):** Verinin aktarım veya depolama sırasında kazara (gürültü, hat kopması) bozulup bozulmadığının tespiti.
2. **Kimlik Doğruluğu ve Yetki (Authenticity):** Yazılımın gerçekten yetkili ve güvenilir kaynak tarafından üretildiğinin kanıtlanması.
3. **Gizlilik (Confidentiality):** Yazılım içeriğinin yetkisiz kişilerin eline geçmesini önlemek amacıyla şifrelenmesi.

Basit hata tespit yöntemlerinin veya sistem popülerliğinin kriptografik bir güvenlik sunduğu yanılgısına düşülmemelidir.

6.2. CRC, Hash, Dijital İmza ve Şifreleme Ayrımları

Yöntem	Matematiksel Mantık	Mekanizma	Sağladığı Güvenlik Boyutu
CRC (Cyclic Redundancy Check / Döngüsel Artıklık Kontrolü)	Polinom bölmesine dayalı matematiksel kontrol.	Veri baytlarından sabit boyutlu bir kontrol değeri üretir.	**Sadece Veri Bütünlüğü** (Kazara oluşan hataları tespit eder; kötü niyetli değiştirmeye karşı korumasızdır).
Hash (Kriptografik Özet)	Tek yönlü matematiksel fonksiyonlar (MD5, SHA-256).	Veriden benzersiz, geri döndürülemez sabit boyutlu özet üretir.	**Veri Bütünlüğü** (En küçük veri değişiminde özet tamamen değişir; tek başına kaynak doğrulamaz).
Digital Signature (Dijital İmza)	Asimetrik Kriptografi (RSA, ECC - Ed25519/ECDSA).	Özel anahtar (Private Key) ile imzalama, Açık anahtar (Public Key) ile doğrulama.	**Veri Bütünlüğü + Kimlik Doğruluğu** (Yazılımın yetkili üreticiden geldiğini garanti eder).
Encryption (Şifreleme)	Simetrik/Asimetrik Algoritmalar (AES-128/256).	Veriyi okunamaz karmaşık metne (ciphertext) dönüştürür.	**Gizlilik** (Fikri mülkiyeti korur, yetkisiz kopyalamayı engeller).

6.3. Secure Boot ve Anti-Rollback Mimarisi

- **Secure Boot (Güvenli Açılış):** İşlemci resetlendiğinde, çalıştırılacak her yazılım katmanının (Bootloader -> Uygulama) kriptografik dijital imzasını doğrulayan ve yalnızca yetkili imzaya sahip kodların yürütülmesine izin veren güvenlik mimarisidir.
- **Anti-Rollback (Eski Sürüm Engelleme):** Kötü niyetli kişilerin, cihazı bilinen güvenlik açıkları barındıran eski bir yazılım sürümüne (downgrade) döndürmesini engelleyen mekanizmadır [S06].

Anti-rollback mekanizması, donanımsal olarak geri döndürülemez **eFuse (Electronic Fuse / Elektronik Sigorta)** saklayıcıları üzerinden yürütülür. Örneğin ESP32 mimarisinde **EFUSE_BLK3_RDATA4_REG** saklayıcısı içinde yakılan bit sayısı cihazın güvenlik sürümünü (**secure_version**) belirler. Bootloader, yeni yazılımın güvenlik sürümü eFuse üzerindeki değerden düşükse önyüklemeyi reddeder.

---

7. Kesintisiz Güncelleme, Slot Mimarisi, Güç Kesintisi ve Güvenli Atlama

7.1. Bellek Bölümleme ve Slot Mimarisi (Single Slot vs. Dual Slot / A-B Slot)

- **Slot (Bellek Alanı/Bölümü):** Kalıcı bellek üzerinde belirli bir yazılım imgesini barındırmak için ayrılmış tanımlı alanlardır.
- **Single-Slot (Tek Alan) Mimarisi:** Mevcut uygulamanın üzerine doğrudan yeni yazılım yazılır. Bellek tasarrufu sağlar ancak yazma anında güç kesilirse cihaz kullanılamaz hale gelebilir (bricked).
- **Dual-Slot / A-B Partition (Çift Alan) Mimarisi:** Bellekte iki ayrı uygulama alanı (**ota_0** ve **ota_1**) tanımlanır [S06]. Cihaz **ota_0** alanındaki yazılımı çalıştırırken, yeni güncelleme **ota_1** alanına indirilir. İndirme ve doğrulama tamamlanmadan aktif yazılım bozulmaz.

7.2. Geri Alma (Rollback) Süreci ve Durum Yönetimi

Çift slotlu mimaride OTA güncelleme süreçleri tanımlı durum makineleri (State Machine) üzerinden yönetilir. ESP-IDF mimarisindeki OTA durumları [S06]:

- **ESP_OTA_IMG_NEW**: Yeni indirilen ve ilk kez önyüklenecek imge.
- **ESP_OTA_IMG_PENDING_VERIFY**: İlk açılışı yapılan, kendi kendine test (self-test) onayını bekleyen imge.
- **ESP_OTA_IMG_VALID**: Kendi kendine testi başarıyla geçmiş ve onaylanmış kararlı imge.
- **ESP_OTA_IMG_INVALID**: Doğrulamadan geçememiş, çalıştırılması yasaklanmış imge.
- **ESP_OTA_IMG_ABORTED**: Onaylanmadan reset gerçekleştiği için iptal edilen imge.

Yeni yazılım ilk açılışta tanı fonksiyonunu (**diagnostic()**) çalıştırır. Test başarılı ise durum **ESP_OTA_IMG_VALID** olarak işaretlenir. Test başarısız olursa veya uygulama onay vermeden sistem çökerse, bootloader durumu **ESP_OTA_IMG_INVALID** yapar ve otomatik olarak çalışan eski kararlı sürüme geri döner (**Rollback**).

```
// ESP-IDF OTA İlk Açılış Doğrulama ve Rollback Karar Psödo Kodu
```

```
#include "esp_ota_ops.h"

void app_main(void) {
    const esp_partition_t *running = esp_ota_get_running_partition();
    esp_ota_img_states_t ota_state;

    if (esp_ota_get_state_partition(running, &ota_state) == ESP_OK) {
        if (ota_state == ESP_OTA_IMG_PENDING_VERIFY) {
            // Kendi kendine tanı/test fonksiyonunu çalıştır
            bool diagnostic_ok = run_system_diagnostics();

            if (diagnostic_ok) {
                // Uygulama kararlı, rollback iptal edilsin
                esp_ota_mark_app_valid_cancel_rollback();
            } else {
                // Tanı başarısız, eski çalışan uygulamaya geri dön ve reset at
                esp_ota_mark_app_invalid_rollback_and_reboot();
            }
        }
    }
    // Normal uygulama akışı devam eder...
}
```

7.3. Güç Kesintisi Senaryoları ve Güvenli Atlama Koşulları (Power Loss Resilience)

Güncelleme sırasında oluşabilecek güç kesintilerine karşı veri bütünlüğünü korumak amacıyla metadata yönetimi atomik olarak yürütülür. Örneğin ESP-IDF'te **otadata** bölümü iki ayrı sektörden (**0x2000** bayt) oluşur [S06]. Sektörler bağımsız olarak silinip yazılır; güç kesintisi anında sektörlerden biri bozulsa dahi dizi sayacı (sequence counter) üzerinden en son geçerli olan metadatanın bulunduğu sektör tespit edilerek sistemin çökmesi engellenir.

Bootloader'ın bir uygulamaya **Güvenli Atlama (Safe Jump)** gerçekleştirebilmesi için aşağıdaki önkoşulların eksiksiz sağlanması gerekir:

1. Geçerli sihirli baytın (magic byte/word) kontrol edilmesi.
2. İmza veya CRC/Hash doğrulamasının başarılı olması.
3. Vektör tablosundaki Yığın Göstergesinin (Stack Pointer) geçerli RAM adres aralığında olması.
4. VTOR register'ının yeni uygulamanın adresine güncellenmesi.

---

8. Somut Donanım ve Platform İncelemeleri

8.1. STM32 Mimarisi ve Ekosistemi

STMicroelectronics STM32 platformunda IAP süreçleri seri haberleşme protokolleri üzerinden yürütülür [S03]. AN3155 uygulama notunda detaylandırılan ST USART Bootloader Protokolü, sıkı tanımlanmış bir el sıkışma (handshake) ve komut yapısına dayanır [S02]:

- **Aksel yanıtlar:** Her geçerli komuta karşılık Boot ROM **0x79** (**ACK** - Acknowledge) yanıtı döndürür; hatalı veya yetkisiz komutlarda **0x1F** (**NACK** - Negative Acknowledge) ile yanıt verir.

- **Temel Komut Kümesi:**
  - **Get** (**0x00**): Desteklenen bootloader sürümünü ve komut listesini sorgular.
  - **Read Memory** (**0x11**): Belirtilen bellek adresinden veri okur.
  - **Go** (**0x21**): Belirtilen bellek adresindeki koda atlayarak yürütmeyi başlatır.
  - **Write Memory** (**0x31**): Belirtilen Flash bellek adresine veri bloğu yazar.

IAP uygulamasında Flash belleğe yazma işlemi öncesinde kilidin açılması (**FLASH_Unlock**), ilgili sektörün silinmesi (**FLASH_PageErase**) ve verilerin kelime bazında yazılması (**FLASH_ProgramWord**) adımları sırayla icra edilir [S03].

8.2. ESP32 ve ESP-IDF Bootloader / OTA Mimarisi

ESP32 mimarisinde çok aşamalı açılış yürütülür [S05]:

- **Birinci Aşama (First Stage - ROM Bootloader):** Çip içerisindeki ROM'dan çalışır. Flash belleğin **0x1000** adresinde yer alan İkinci Aşama Bootloader'ı RAM'e yükler.
- **İkinci Aşama Bootloader (Second Stage Bootloader):** Flash belleğin **0x8000** adresindeki Bölüm Tablosunu (**Partition Table**) ve **otadata** sektörlerini okur. Hangi uygulama slotunun (**ota_0** veya **ota_1**) başlatılacağını seçer.

ESP32 ekosisteminde kilit bileşenler:

- **RTC Watchdog (RTC_WDT):** RTC (Real-Time Clock / Gerçek Zamanlı Saat) Watchdog Timer (**RTC_WDT**), bootloader aşamasında varsayılan olarak 9 saniyelik bir zaman aşımı süresine sahiptir [S05]. Uygulama 9 saniye içinde kararlı şekilde başlatılamazsa çip resetlenir.
- **Deep-Sleep Hızlı Açılış:** Derin uyku modundan uyanırken doğrulama adımlarını atlayarak açılış süresini ve güç tüketimini azaltmak için RTC FAST belleğinde tutulan adres kullanılır (**CONFIG_BOOTLOADER_SKIP_VALIDATE_IN_DEEP_SLEEP**) [S05].
- **otatool.py:** ESP-IDF tarafından sunulan, CLI (Command Line Interface / Komut Satırı Arayüzü) ve Python API (Application Programming Interface / Uygulama Programlama Arayüzü) üzerinden **otadata** okuma, silme, slot değiştirme veya manuel OTA yazma işlemlerini gerçekleştiren araçtır [S06].

> **Secure Boot V2 Sınır Detayı:** Secure Boot V2 etkinleştirildiğinde ikinci aşama bootloader için tanımlanan 48 KB (**0xC000** bayt) ikili boyut sınırı, dosya sonuna eklenen **4 KB imza bloğunu kapsamaz**. Dolayısıyla bootloader'ın Flash üzerinde kapladığı toplam fiziksel alan 48 KB + 4 KB imza = **52 KB** olmaktadır [S05].

8.3. MCUboot Projesi Özellikleri

MCUboot, 32-bit mikrodenetleyiciler için geliştirilmiş, açık kaynaklı standart bir bootloader projesidir [S04]. MCUboot mimarisinde imge başlıkları, uygulama kodları ve imge sonundaki izleme alanı (**Trailer Structure**) yer alır.

- **Trailer Yapısı ve Durum Bayrakları:** Bellek alanının sonunda yer alan trailer; imge doğrulama durumunu, takas adımlarını ve **copy_done**, **image_ok** gibi durum bayraklarını barındırır.
- **Çalışma Modları:**

- **Swap Scratch:** İki slot arasındaki imge verilerini geçici bir çizim alanı (scratch partition) kullanarak takas eder.
- **Direct-XIP:** Takas yapmadan, doğrulanmış imgeyi bulunduğu slot üzerinden doğrudan çalıştırır.
- **Overwrite-Only:** Eski imgenin üzerine yenisini doğrudan yazar (geri alma desteği sunmaz).

8.4. Akademik Çalışma Analizi: UpKit Çerçevesi (Langiu et al., ICDCS 2019)

Langiu ve arkadaşları tarafından sunulan "UpKit: An Open-Source, Portable, and Lightweight Update Framework for Constrained IoT Devices" başlıklı akademik çalışma, kısıtlı kaynaklara sahip IoT (Internet of Things / Nesnelerin İnterneti) cihazları için uçtan uca bir güncelleme çerçevesi sunmaktadır [S08].

- **Temel Katkıları:** Güncellemenin tüm aşamalarını (üretim, imzalama, iletim, doğrulama, kurulum) kapsar. Çift imzalama (double-signature) süreci ile verinin tazeliğini (freshness) garanti eder. İndirme hemen sonrasında, henüz bootloader aşamasına geçilmeden erken doğrulama (early post-download verification) yaparak hatalı imgenin cihaza yüklenmesini ve gereksiz reboot atılmasını engeller. Farklı donanım/OS platformlarına taşınabilirdir (portable) ve fark güncellemelerini (differential updates) destekler.
- **Akademik Yaygınlık ve Değerlendirme Metodolojisi:** UpKit çalışması, OpenAlex veritabanı kayıtlarına göre (4 Ekim 2026 kontrolü itibarıyla) **29 atıf** almıştır (DOI: **10.1109/icdcs.2019.00207**) [S08]. Akademik literatürde atıf sayıları indeksleme kapsamına, makalenin yayınlandığı yıla ve disipline göre değişiklik gösterir. Atıf sayısı tek başına mutlak bir teknik üstünlük puanı olmayıp, çalışmanın akademik ekosistemdeki görünürlüğünü ifade eder.

---

9. Sayısal Örnekler ve Mühendislik Hesaplama Adımları

9.1. Sayısal Örnek 1: ESP32 Bootloader Boyutu ve Partition Table Offset Hesaplaması

Problem/Senaryo: Varsayılan konfigürasyonda Partition Table Offset adresi **CONFIG_PARTITION_TABLE_OFFSET = 0x8000** (32.768 bayt / 32 KB) olarak tanımlanmıştır [S05]. ESP32 İkinci Aşama Bootloader ikili dosyası Flash üzerinde **0x1000** (4.096 bayt / 4 KB) adresinden başlamaktadır. Secure Boot V2 ve ek özelliklerin etkinleştirilmesi sonucu bootloader ikili dosya boyutu 30 KB (**0x7800** bayt) seviyesine ulaşmıştır.

Adım 1: Varsayılan Kullanılabilir Alan Hesaplaması

Kullanılabilir Bootloader Alanı = CONFIG_PARTITION_TABLE_OFFSET - 0x1000

Kullanılabilir Alan = 0x8000 - 0x1000 = 0x7000 bayt

0x7000 bayt = 28.672 bayt = 28 KB

Adım 2: Hata Durumunun Değerlendirilmesi Derlenen yeni bootloader boyutu: **30 KB = 30.720 bayt (0x7800 bayt)**.

0x7800 bayt > 0x7000 bayt ⇒ Limit Aşıldı!

Bu durumda derleme sistemi "*Bootloader binary size is too large for partition table offset*" hatası verecektir. Cihaz bu haliyle programlanırsa Partition Table verisinin üzeri yazılacağı için çip açılmayacaktır.

Adım 3: Çözüm Hesaplaması (Partition Table Offset'inin Kaydırılması) Gerekli alan **0x7800** bayt olduğuna göre, yeni offset değeri en az:

Yeni Minimum Offset = 0x1000 + 0x7800 = 0x8800

Sektör hizalaması (4 KB / **0x1000** katları) kuralı gereğince **CONFIG_PARTITION_TABLE_OFFSET** değeri **0x9000** veya güvenli marj için **0xA000** adresine çekilmelidir [S05].

> **Sınır Şartı:** Secure Boot V2 etkinleştirildiğinde, bootloader doğrulama tamponunun sabit boyutu nedeniyle bootloader ikili boyutu için 48 KB (**0xC000** bayt) mutlak ikili boyutu üst sınırı bulunmaktadır (4 KB imza bloğu hariç toplam 52 KB) [S05].

9.2. Sayısal Örnek 2: Flash Sektör Silme Zamanı ve Watchdog Timeout Hesabı

Problem/Senaryo: ESP32 bootloader bünyesindeki RTC Watchdog (**RTC_WDT**) varsayılan zaman aşımı süresi **9000 ms (9 saniye)** olarak ayarlanmıştır [S05]. Tasarlanan cihazda 1 MB (**1024 KB**) boyutundaki bir uygulama bölümünün (partition) toplu olarak silinmesi (bulk erase) gerekmektedir. Kullanılan Flash belleğin silme hızı her 4 KB sektör başına ortalama **100 ms**'dir.

Adım 1: Toplam Silme Süresinin Hesaplanması

1 MB = 1024 KB

Sektör Sayısı = (1024 KB)/(4 KB) = 256 sektör

Toplam Silme Süresi = 256 × 100 ms = 25.600 ms = 25,6 saniye

Adım 2: Watchdog Çatışmasının Analizi

Hesaplanan Silme Süresi (25,6 s) > RTC_WDT Zaman Aşımı (9 s)

Silme işlemi sırasında işlemci Flash blokajı nedeniyle RTC Watchdog zamanlayıcısını besleyemeyecek (feed/reset yapamayacak) ve 9. saniyede RTC_WDT çipi resetleyecektir. Bu durum sonsuz bir reset döngüsüne (reset loop) neden olur.

Adım 3: Mühendislik Çözümü

1. Silme işlemini sıralı/kademeli (incremental erase) olarak küçük bloklar halinde yapmak ve aralarda Watchdog'u beslemek.
2. Veya **CONFIG_BOOTLOADER_WDT_TIME_MS** parametresini artırarak (örneğin 30.000 ms yapılarak) derlemek [S05].

---

10. Yaygın Hatalar ve Sorun Giderme Rehberi (Troubleshooting)

10.1. Mühendislik Tasarımında Sık Yapılan Hatalar

- **CRC ile Kriptografik Güvenliği Karıştırmak:** Basit bir CRC32 kontrolünü yazılımın yetkili üreticiden geldiğini kanıtlamak için kullanmak (CRC kötü niyetli değiştirmeleri engellemez).
- **Atomik Olmayan Metadata Güncellemesi:** Güç kesintisi anında durum göstergelerini tek bir bellek hücresine korumasız yazmak.
- **VTOR Relokasyonunu Unutmak:** Bootloader'dan uygulamaya atlarken Vektör Tablosu Offset Saklayıcısını uygulamanın başlangıç adresine yönlendirmemek.
- **Çevre Birimlerini Sıfırlamadan Atlama Yapmak:** Bootloader içinde aktif kullanılan DMA veya kesme kaynaklarını kapatmadan doğrudan uygulama koduna atlamak (uygulama açılışında çökmeye neden olur).

10.2. Sorun Giderme Matrisi (Troubleshooting Table)

Belirti / Hata Mesajı	Olası Neden	Çözüm ve Düzeltme Adımı
"*Bootloader binary size is too large for partition table offset*"	Bootloader ikili boyutunun **CONFIG_PARTITION_TABLE_OFFSET - 0x1000** alanını aşması [S05].	Compiler optimizasyonunu "Size" yapın, Log seviyesini düşürün veya Partition Table Offset değerini yükseltin (**0x9000**) [S05].
Açılışta Sürekli RTC_WDT Reset Döngüsü	Flash silme veya başlatma süresinin 9 saniyelik RTC Watchdog süresini aşması [S05].	**CONFIG_BOOTLOADER_WDT_TIME_MS** değerini artırın veya toplu silme yerine kademeli silme yapın [S05, S06].
ESP_ERR_OTA_ROLLBACK_INVALID_STATE	**ESP_OTA_IMG_PENDING_VERIFY** durumundaki yazılım onaylanmadan yeni OTA başlatılmaya çalışılması [S06].	Yeni güncelleme indirmeden önce **esp_ota_mark_app_valid_cancel_rollback()** çağırarak mevcut uygulamayı doğrulayın [S06].
Açılışta Anti-Rollback Reddi / Boot Engeli	Uygulamanın **secure_version** değerinin eFuse içindeki değerden küçük olması [S06].	Uygulamanın **secure_version** değerini eFuse'taki değere eşit veya daha yüksek olacak şekilde güncelleyip yeniden derleyin [S06].

---

11. Kapsamlı Kavram ve Kısaltma Sözlüğü (Glossary)

Kısaltma / Terim	İngilizce Açılımı ve Türkçe Karşılığı	Teknik Açıklama ve Görevi
Firmware	Embedded Software / Gömülü Cihaz Yazılımı	Donanım işlevlerini doğrudan kontrol eden alt seviye yazılımdır.

Kısaltma / Terim	İngilizce Açılımı ve Türkçe Karşılığı	Teknik Açıklama ve Görevi
Bootloader	Boot Loader / Önyükleyici	Sistem açılışında sonraki yazılımı seçen, doğrulayan ve başlatan önyükleyici koddur.
CPU	Central Processing Unit / Merkezi İşlem Birimi	Komutları yürüten temel işlem çekirdeğidir.
MCU	Microcontroller Unit / Mikrodenetleyici	CPU, bellek ve çevre birimlerini tek çipte toplayan entegre devredir.
ROM	Read-Only Memory / Salt Okunur Bellek	İçeriği değiştirilemeyen kalıcı bellek birimidir.
Flash	Flash Memory / Elektriksel Silinebilir Kalıcı Bellek	Sektörler halinde silinip yazılabilen kalıcı bellek teknolojisidir.
RAM	Random Access Memory / Rastgele Erişimli Çalışma Belleği	Değişkenlerin ve geçici verilerin tutulduğu uçucu çalışma belleğidir.
ELF	Executable and Linkable Format / Yürütülebilir ve Bağlanabilir Dosya Biçimi	Hata ayıklama ve sembol verileri içeren karmaşık derleme çıktı formatıdır.
BIN	Binary File / Ham İkili Dosya	Adres bilgisi içermeyen ham makine kodu dizisidir.
Intel HEX	Hexadecimal Text Format / Onaltılık Metin Biçimi	Adres ve checksum bilgileri barındıran ASCII tabanlı kodlama biçimidir.
Linker	Linker / Bağlayıcı	Amaç dosyalarını birleştirip bellek haritasına yerleştiren derleme aracıdır.
Memory Map	Memory Map / Bellek Haritası	Donanım bellek adreslerinin kullanım düzenini gösteren şemadır.
Vector Table	Vector Table / Vektör Tablosu	Kesme ve reset işleyicilerinin adreslerini tutan tablodur.
Stack	Stack / Yığın	Geçici çalışma verilerini ve fonksiyon dönüş adreslerini saklayan RAM alanıdır.
Reset Vector	Reset Vector / Reset Vektörü	İşlemcinin reset sonrası çalıştıracağı ilk komutun adresidir.
Startup Code	Startup Code / Başlangıç Kodu	C çalışma ortamını ve RAM alanlarını hazırlayan ilk kod bloğudur.
UART	Universal Asynchronous Receiver/Transmitter / Asenkron Seri Alıcı/Verici	Asenkron seri haberleşme çevre birimidir.
USART	Universal Synchronous/Asynchronous Receiver/Transmitter / Senkron-Asenkron Seri Birim	Senkron veya asenkron çalışabilen gelişmiş seri haberleşme birimidir.

Kısaltma / Terim	İngilizce Açılımı ve Türkçe Karşılığı	Teknik Açıklama ve Görevi
USB	Universal Serial Bus / Evrensel Seri Veri Yolu	Yüksek hızlı evrensel seri bağlantı standardıdır.
SPI	Serial Peripheral Interface / Seri Çevre Birimi Arabirimi	Tam çift yönlü senkron seri haberleşme protokolüdür.
SWD	Serial Wire Debug / Seri Tel Hata Ayıklama	Arm mimarilerinde kullanılan iki hatlı hata ayıklama arayüzüdür.
JTAG	Joint Test Action Group / Ortak Test Eylem Grubu	Donanımsal test ve programlama arayüzü standardıdır.
IAP	In-Application Programming / Uygulama İçinden Programlama	Çalışan uygulamanın kalıcı belleği güncellemesidir.
OTA	Over-The-Air / Kablosuz-Uzaktan Güncelleme	Güncelleme paketlerinin kablosuz ağlar üzerinden aktarılmasıdır.
Slot	Partition Slot / Bellek Alanı	Yazılım görselleri için ayrılmış tanımlı Flash bölümüdür.
Rollback	Rollback / Geri Alma	Hatalı güncelleme sonrası önceki çalışan sürüme dönme işlemidir.
Anti-Rollback	Anti-Rollback / Eski Sürüm Engelleme	Güvenlik açığı barındıran eski yazılımlara dönüşü engelleme mekanizmasıdır.
CRC	Cyclic Redundancy Check / Döngüsel Artıklık Kontrolü	Veri bozulmalarını tespit eden non-kriptografik kontrol yöntemidir.
Hash	Cryptographic Hash / Kriptografik Özet	Veriden benzersiz sabit boyutlu özet üreten tek yönlü fonksiyondur.
Digital Signature	Digital Signature / Dijital İmza	Kaynak yetkisini ve bütünlüğü doğrulayan asimetrik kriptografik imzadır.
Encryption	Encryption / Şifreleme	Veri gizliliğini sağlamak için içeriği okunamaz hale getirme işlemidir.
Secure Boot	Secure Boot / Güvenli Açılış	Yalnızca yetkili imzaya sahip yazılımların çalıştırılmasını sağlayan mimaridir.
MPU	Microprocessor Unit / Mikroişlemci Birimi	Harici bellek kullanan yüksek güçlü uygulama işlemcisidir.
OS	Operating System / İşletim Sistemi	Donanım kaynaklarını ve süreçleri yöneten yazılım katmanıdır.
Linux	Linux Kernel / Linux Çekirdeği	Açık kaynaklı gelişmiş işletim sistemi çekirdeğidir.
U-Boot	Das U-Boot / Evrensel Önyükleyici	MPU ve Linux sistemlerinde kullanılan gelişmiş 3. aşama bootloader'dır.

Kısaltma / Terim	İngilizce Açılımı ve Türkçe Karşılığı	Teknik Açıklama ve Görevi
DTB	Device Tree Blob / Aygıt Ağacı İkili Bloğu	Donanım konfigürasyonunu çekirdeğe bildiren ikili veri dosyasıdır.
FIT	Flattened Image Tree / Düzleştirilmiş Görüntü Ağacı	Çekirdek, DTB ve overlays verilerini tek dosyada toplayan formattır.
eFuse	Electronic Fuse / Elektronik Sigorta	Tek seferlik programlanabilen (OTP) donanımsal sigorta hücresidir.
ESP-IDF	Espressif IoT Development Framework / Espressif IoT Geliştirme Çatısı	ESP32 serisi çipler için resmi geliştirme yazılım ortamıdır.
IoT	Internet of Things / Nesnelerin İnterneti	İnternete bağlı akıllı cihazlar ekosistemidir.

---

12. Değerlendirme ve Kontrol Soruları (Açıklamalı Yanıtlar)

1. **Bootloader bir firmware midir? Aralarındaki temel işlevsel fark nedir?**
   - *Yanıt:* Evet, bootloader özelleşmiş bir firmware türüdür (önyükleyici firmware). Temel fark; uygulama firmware'inin cihazın nihai görevlerini (sensör, motor vb.) yürütmesine karşın, bootloader'ın yalnızca açılışta sistemi hazırlamak, doğrulamak ve ana uygulamayı başlatmakla görevli olmasıdır.
2. **BIN dosyası yüklenirken neden başlangıç adresi belirtilmelidir, Intel HEX dosyasında bu neden gerekmez?**
   - *Yanıt:* BIN dosyaları hiçbir adres bilgisi barındırmayan ham bayt dizileridir. Intel HEX dosyaları ise ASCII formatında satır bazlı hedef bellek adres kayıtlarını bünyesinde barındırır.
3. **Bootloader'dan uygulamaya atlarken Vector Table Relocation (VTOR) yapılmazsa ne olur?**
   - *Yanıt:* İşlemci bir kesme (interrupt) aldığında varsayılan **0x00000000** adresindeki vektör tablosuna bakar. VTOR güncellenmezse, uygulamanın kesme işleyicileri yerine bootloader'ın kesme işleyicileri çağrılır ve sistem çöker.
4. **Fabrika ROM yükleyicisi ile Flash üzerindeki özel uygulama yükleyicisi arasındaki temel farklar nelerdir?**
   - *Yanıt:* Fabrika ROM yükleyicisi silinemez/değiştirilemez ve üretici tarafından konmuştur. Özel uygulama önyükleyicisi ise geliştirici tarafından Flash'a yazılır, güncellenebilir ve özel şifreleme/slot mantıkları barındırabilir.
5. **IAP ve OTA güncellemelerinde CRC kullanımı cihazın yetkili yazılım çalıştırdığını kanıtlar mı? Neden?**
   - *Yanıt:* Hayır kanıtlamaz. CRC yalnızca iletim sırasındaki kazara veri hatalarını tespit eder. Kötü niyetli bir kişi yazılımı değiştirip CRC değerini de yeniden hesaplayabilir. Yetki doğrulaması için Dijital İmza (Digital Signature) gereklidir.
6. **Rollback ile Anti-Rollback arasındaki mantıksal ve teknik fark nedir?**

- *Yanıt:* Rollback, yeni yazılımın çökmesi durumunda sistemin çalışır durumdaki bir önceki sürüme otomatik dönmesidir (güvenilirlik mekanizması). Anti-Rollback ise bilinen açıkları olan eski yazılımlara geri dönülmesini eFuse sigortaları ile engelleyen güvenlik mekanizmasıdır.

7. **ESP32'de otadata bölümünün iki ayrı sektörden (sector) oluşmasının temel donanımsal nedeni nedir?**
   - *Yanıt:* Güç kesintisi korumasıdır. Sektörler sırayla silinip yazılır. Yazma anında güç kesilirse diğer sektördeki veriler ve sıra sayacı (sequence counter) korunarak sistemin çökmesi engellenir.
8. **ESP_OTA_IMG_PENDING_VERIFY durumundaki bir uygulama doğrulanmadan reboot edilirse ne gerçekleşir?**
   - *Yanıt:* Bootloader bu uygulamanın durumunu **ESP_OTA_IMG_ABORTED** olarak değiştirir ve bir önceki geçerli uygulamaya geri döner (rollback gerçekleştirir).
9. **UpKit çalışmasının gömülü yazılım güncelleme literatürüne getirdiği en temel yenilik nedir?**
   - *Yanıt:* İndirme hemen sonrasında (erken aşamada) doğrulamayı yapması, cihazın gereksiz yere reboot edilmesini önlemesi ve çift imzalama ile veri tazeliğini garanti etmesidir [S08].
10. **Secure Boot V2 aktifken bootloader ikili dosya boyut üst sınırı neden 48 KB (0xC000 bytes) ile sınırlıdır?**
    - *Yanıt:* Bootloader'ın doğrulanması esnasında yüklendiği sabit boyutlu bellek tamponunun sınırlı olması nedeniyle bu üst sınır mevcuttur [S05]. (4 KB imza bloğu hariç toplam alan 52 KB'dir).
11. **U-Boot üzerinde bootcmd değişkeninin temel görevi nedir?**
    - *Yanıt:* Otomatik önyükleme (autoboot) süresi dolduğunda çalıştırılacak varsayılan komut dizisini (örneğin bellekten çekirdek okuma ve **bootm** çalıştırma) barındırmaktır [S07].
12. **Gömülü sistemlerde dosya boyutunun küçük olması RAM kullanımının da düşük olacağını garanti eder mi?**
    - *Yanıt:* Hayır. Dosya boyutu Flash kaplamasını gösterir. RAM kullanımı dinamik değişkenler, yığın boyutu ve **.bss**/**.data** bölümlerine bağlıdır.
13. **SWD/JTAG ile UART bootloader protokolü arasındaki temel fark nedir?**
    - *Yanıt:* SWD/JTAG donanımsal çip erişim ve hata ayıklama protokolüdür. UART bootloader ise seri hat üzerinden çalışan yazılımsal bir iletişim protokolüdür.
14. **ESP32'de RTC_WDT süresi varsayılan olarak kaç saniyedir ve aşılırsa ne olur?**
    - *Yanıt:* Varsayılan süre 9 saniyedir. Süre aşılırsa sistem RTC Watchdog tarafından otomatik olarak resetlenir [S05].
15. **FIT (Flattened Image Tree) dosyasının geleneksel uImage dosyasına göre avantajı nedir?**
    - *Yanıt:* İşletim sistemi çekirdeği, Aygıt Ağacı (DTB) ve Device Tree Overlays (DTBO) yapılarını tek bir bütünleşik paket altında toplamasıdır [S07].

---

13. Kaynakça ve Kaynak Sınırlamaları (References & Source Limitations)

13.1. Kullanılan Kaynaklar Listesi

- **[S01]** STMicroelectronics, *AN2606 Application Note: STM32 microcontroller system memory boot mode*.
- **[S02]** STMicroelectronics, *AN3155 Application Note: USART protocol used in the STM32 bootloader*.
- **[S03]** STMicroelectronics, *AN4657 Application Note: STM32 in-application programming (IAP) using the USART*.
- **[S04]** MCUboot Project, *MCUboot Bootloader Design and Documentation*.
- **[S05]** Espressif Systems, *ESP-IDF Programming Guide v6.1: Bootloader*.
- **[S06]** Espressif Systems, *ESP-IDF Programming Guide v6.1: Over The Air Updates (OTA)*.
- **[S07]** Microchip Technology / Linux4SAM, *U-Boot for AT91 / SAMA SoCs Documentation*.
- **[S08]** A. Langiu, C. A. Boano, M. Schuß, K. Römer, "UpKit: An Open-Source, Portable, and Lightweight Update Framework for Constrained IoT Devices," *IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*, 2019. (OpenAlex ID: **W2982540289**, DOI: **10.1109/icdcs.2019.00207**).

13.2. Kaynak Sınırlamaları ve Metodolojik Beyan

Bu teknik rapor, yalnızca sağlanan kaynak metinlerde (**Source Context: S01 - S08**) yer alan gerçekler ve sınırlar çerçevesinde hazırlanmıştır. Kaynaklarda yer almayan donanım spesifikasyonları, pin numaraları ve register düzeyindeki assembly kod detayları kurgulanmamış ve rapora dahil edilmemiştir. İncelemelerde sunulan atıf metrikleri (örneğin UpKit makalesinin 29 atıfı), veritabanlarının 4 Ekim 2026 tarihi itibarıyla indeksleme kapsamını yansıtmakta olup mutlak bir kalite sıralaması olarak değerlendirilmemelidir.